GC type	Threading model	Pause time	Heap size support	Throughput	Fragmentation handling	Recommended for	Concurrent compaction
Serial GC	Single thread	High (stop-the-world)	Small	Low	Poor	Small apps, single-core systems	No
Parallel GC	Multi-threaded	Moderate	Medium-Large	High	Moderate	Batch processing, multicore systems	No
CMS GC	Multi-threaded (concurrent)	Low	Medium-Large	Moderate	Poor (fragmentation possible)	Low-latency apps (deprecated)	No
G1 GC	Multi-threaded (concurrent + parallel)	Configurable (low to moderate)	Large	High	Good (region-based)	General purpose, latency-sensitive apps	Partial
Epsilon GC	Single thread	None (no collection)	Small-Medium	Highest (no GC overhead)	N/A	Testing, custom memory management	No
ZGC	Multi-threaded (concurrent)	Very low (<2ms)	Huge (>4TB)	High	Excellent	Large heaps, real-time, cloud	Yes
Shenandoah GC	Multi-threaded (concurrent)	Very low (<10ms)	Huge (>4TB)	High	Excellent	Large heaps, low latency	Yes

requiring minimal pause times and large heaps.

Command-line usage:

```
java -XX:+UseZGC -jar test-application-name.jar
```

- ***Shenandoah Garbage Collector***

Recommended scenario: This GC is suitable for low-latency applications with large heaps.

Command-line usage:

```
java -XX:+UseShenandoahGC -jar test-application-name.jar
```

- ***Epsilon Garbage Collector***

Recommended scenario: Epsilon is suitable for performance testing and benchmarking scenarios where memory reclamation is not required.

Command-line usage:

```
java -XX:+UseEpsilonGC -jar test-application-name.jar
```

The table above summarises the key characteristics of each garbage collector discussed above. This should help you match a collector to your specific application needs.

Java's garbage collectors have evolved to meet a wide variety of demands, from tiny embedded systems to sprawling cloud services that require heaps spanning terabytes. Java 25 continues this tradition, refining the latest collectors to be even more responsive and powerful. As you select a GC for your next Java project, weigh your application's needs for performance, pause time, and heap size support, and don't forget to keep an eye on the latest improvements—these can make a real difference in your application's stability and responsiveness. END

By: Dr Magesh Kasthuri

The author is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd.

Disclaimer: *This article expresses his views and not of the organisation he works in.*